PS18 Master Project Report — One Front Door for Everything

Complete reference for AI-assisted development

Purpose of this document: this is meant to be pasted into an AI coding assistant's context (Claude Code, Cursor, or similar) before work starts, so it understands the full system, not just the file it's currently looking at. It's self-contained, everything needed to reason about the project correctly is in here.

---

1. Problem Statement

Users need help from IT, HR, Finance, and Facilities but don't know which of a dozen separate bots to ask, so they give up. The fix is one conversational front door backed by a routing and assignment system: a question gets classified by department, answered directly when the system is confident and has grounded evidence, or handed to the right human when it isn't. Success is measured by routing accuracy and true end-to-end resolution, not just "the bot replied."

---

2. Goals and Non-Goals (MVP scope)

Build for real:

- IT and HR departments with real knowledge base content
- Router with confidence gate and one-question clarification
- Ticket creation, assignment (skill match + load only), and manual status updates
- Employee inbox and user chat, connected through one shared thread

Explicitly out of scope for MVP (stretch only, don't let these block the core loop):

- Finance and Facilities can be stubbed with placeholder KB content
- Full automatic tier promotion/demotion math, hardcode tiers for the demo, describe the algorithm rather than running it live
- Predictive SLA-breach reassignment, incident correlation, Teams/Graph integration
- Compliance-grade audit logging, a basic log is enough for a demo

If the team is deciding what to cut under time pressure, cut from the stretch list before touching anything in "build for real."

---

3. Glossary

Term	**Meaning**
Department	IT, HR, Finance, or Facilities, each with its own KB, staff, and manager
Tier	Employee access level (L1/L2/L3), gates which actions an account can perform
Ticket	A request handed from the bot to a human, with transcript, classification, priority, SLA deadline
Confidence gate	The threshold logic deciding auto-answer vs. clarify vs. hand off
Rolling score	An employee's trailing performance metric, drives tier changes
Orchestrator	The LLM layer that reads the message, classifies, answers, and calls tools
KB (knowledge base)	Department-scoped documents the bot retrieves answers from, always cited
Escalation	Moving a ticket up a tier or to a manager when it isn't resolved
CSAT	User satisfaction rating collected after a ticket resolves

---

4. System Architecture

User message
  |
  v
Orchestrator (LLM) --- calls tools ---> Backend services

```
|                                  |
+-- confident, single topic -----> search_knowledge_base -> cited answer
+-- needs a human ----------------> create_ticket -> assignment engine -> employee
                                                          |
                                                  resolved, or escalates
```

Two layers, deliberately separated:

- **Understanding layer (LLM):** classification, clarification, answer generation, tool calls. It never decides who gets a ticket.
- **Business logic layer (plain code):** assignment scoring, tier changes, SLA timers. Deterministic and auditable on purpose, "why did this go to this person" needs a real number, not a model's guess.

---

5. Suggested Tech Stack

Pick one path per team comfort, don't mix without reason.

Layer	Option	Notes
Frontend	Next.js + React + Tailwind	One app, two route groups: `/chat` (user) and `/inbox` (employee)
Backend	Node.js (Fastify/Express) or Python (FastAPI)	Node if the frontend team wants to share types; FastAPI if the team is stronger in Python
Database	PostgreSQL	Tickets, employees, departments, all relational data
Vector search	pgvector (extension on the same Postgres)	Simplest for a hackathon, avoids standing up a separate vector DB
Realtime	Socket.io / WebSockets	Powers the live inbox and the "employee reply appears in user's chat" behavior
LLM	We have to create our Own LLM from Scratch (Research)	Orchestrator calls the tool schema in Section 9

Layer	Option	Notes
Auth	A lightweight library (Auth.js / Clerk)	Don't hand-roll auth under time pressure
Hosting	Vercel (frontend) + Render/Railway (backend) + Supabase or Neon (Postgres)	Fast to deploy, minimal ops overhead

---

6. Data Model

```
User {
  id, name, role, department_scope
}

Employee {
  id, department, tier, skill_tags[],
  status, current_load, rolling_score,
  tenure, certifications[]
}

Ticket {
  id, user_id, transcript, department, tags,
  priority, required_tier, status,
  assigned_to, reassignment_count,
  sla_deadline, created_at, resolved_at
}

Department {
  id, name, manager_id, knowledge_base_id, sla_policy
}

KBArticle {
  id, department_id, title, content, source_citation, embedding
}

FeedbackLog {
  id, ticket_id, csat_score, reopened, comment
}

RoutingLog {
```

  id, message_id, predicted_department, confidence,
  corrected_department (nullable)
}

`RoutingLog` and `FeedbackLog` are the two tables that make the "gets smarter over time" claim real, without them there's no data to measure routing accuracy or retrain anything.

---

7. API Contracts

Method	**Path**	**Purpose**
POST	`/chat/message`	User sends a message, orchestrator responds (answer, clarifying question, or ticket confirmation)
GET	`/tickets/:id`	Fetch ticket detail and current status
POST	`/tickets`	Create a ticket (called by the orchestrator's `create_ticket` tool)
POST	`/tickets/:id/assign`	Run the assignment algorithm against an existing ticket
PATCH	`/tickets/:id/status`	Employee updates status (in progress / waiting / resolved / escalate)
GET	`/employees/:id/inbox`	List of tickets assigned to this employee, sorted by SLA urgency
PATCH	`/employees/:id/availability`	Toggle online/busy/offline, feeds the assignment engine directly
GET	`/kb/search`	Query a department's knowledge base, returns passage + citation

Method	Path	Purpose
POST	`/feedback`	Submit CSAT after a resolution
GET	`/departments/:id/dashboard`	Manager view: open tickets, SLA aging, load distribution

---

8. User System

One continuous thread from first question to resolution. The user sees a cited answer, a single clarifying question, or a handoff notice, never a raw ticket ID. Employee replies land as new messages in that same thread. Reopening is just replying "this didn't fix it." Users never see who specifically is assigned beyond a first name, and data access is scoped to their own role, an HR answer only ever reflects that user's own record.

9. Employee System

A queue, not a conversation. Inbox sorted by SLA urgency. Ticket detail carries the full bot transcript so nobody starts from zero. Status moves through acknowledged → in progress → waiting on user → resolved / escalate. Availability toggle must be wired into the assignment engine for real. Access to actions (not just information) is gated by tier, and a tier change from the rolling score only affects new assignments, not tickets already in progress.

10. Orchestrator (system prompt, condensed)

You are the front-door assistant. Route every question to IT, HR, Finance, or Facilities. Split multi-topic messages into separate handled parts. Never answer a factual question without calling search_knowledge_base first, and always cite the source. If confidence is medium, ask exactly one clarifying question. If confidence is low, or the action needs a permission you don't have, call create_ticket instead of guessing. Never perform or claim to perform a sensitive action without verify_identity succeeding first. Never decide who a ticket is assigned to, call score_and_assign and report its result. Keep responses short and direct.
Full tool schema (search_knowledge_base, create_ticket, score_and_assign, check_ticket_status, verify_identity) is in `PS18_Team_Sprint_Plan_and_Orchestrator_Scaffold.md`, paste-ready as-is.

11. Assignment Algorithm

score(e, t) = w1·skill_match + w2·availability + w3·(1 − load_ratio)
+ w4·historical_success_rate − w5·avg_response_time

Candidates: employees in the ticket's department, at or above the required tier, available. Top 3 scorers go into a weighted random pick, not a strict "always the top score," so the algorithm doesn't overload its best performer. Empty candidate pool → ticket drops into a shared unclaimed pool for that department.

12. Escalation Logic

Triggers: SLA timer expiry, employee flags "outside my access," user reopens a resolved ticket, or reassignment count exceeds 2. First two reassignments bump the ticket to a higher required tier and re-run assignment. Beyond that, it goes straight to the department manager, no more automatic scoring.

---

13. Worked Example Conversations

A — High confidence, resolved directly

> User: "What's our WFH policy?" System: classifies HR, confidence 0.91, retrieves KB passage, replies: "Up to 2 remote days per week with manager approval. [Source: HR Policy §4.2]"

B — Medium confidence, one clarifying question

> User: "It's not letting me in." System: confidence 0.55 between IT (login) and Facilities (badge access). Replies: "Just to confirm, is this your computer login or your building badge?"

C — Multi-topic in one message

> User: "My VPN keeps dropping and I also need to check my leave balance." System: splits into two clauses, classifies IT (high confidence, low retrieval confidence → ticket created) and HR (high confidence → answered directly with citation), both handled in the same reply.

D — Escalation on SLA breach

> Ticket sits untouched past its 2-hour SLA. System auto-bumps required tier from L1 to L2, reassigns, and messages the user: "This is taking longer than expected, I've moved it to a senior specialist. New expected time: 1 hour."

---

14. Non-Functional Requirements

- **Security:** identity verification required before any sensitive action (password reset, refund, payroll change), regardless of routing confidence.
- **Privacy:** users only ever see their own data; employees see only what their department and tier require. Redact PII from logs where not needed.
- **Performance targets (demo-reasonable):** bot answer under 3 seconds, routing decision under 1 second.
- **Auditability:** every routing decision and assignment logs its score/confidence so "why did this happen" is always answerable with a number.

---

15. Metrics and Success Criteria

- Routing accuracy (audited sample, or reassignment rate as a live proxy)
- End-to-end resolution rate, bot-only vs. bot-plus-human
- Average resolution time by tier and department
- Reopen rate and CSAT

---

16. Suggested Repo Structure

```
/frontend
  /app/chat          # user-facing chat
  /app/inbox         # employee queue and ticket detail
  /app/manager       # department dashboard
/backend
  /orchestrator      # system prompt, tool definitions, LLM calls
  /services
    ticket.ts
    employee.ts
    assignment.ts
    escalation.ts
    kb.ts
  /db                # schema, migrations
/shared
  /types             # Employee, Ticket, Department shared across front/back
/docs                # this report + the other three PS18 docs
```

---

17. Team Roles

Track	**Owns**
Orchestrator/AI	System prompt, tool wiring, KB retrieval, citation logic
Backend	Ticket, employee, assignment, escalation services, API contracts
User frontend	Chat UI, ticket-in-thread rendering
Employee frontend	Inbox, status controls, manager dashboard

If under 4 people, merge backend into whichever frontend track has slack, don't split it three ways.

---

18. Risks and Mitigations

Risk	**Mitigation**
LLM answers confidently from general knowledge instead of the KB	Hard rule: no answer without a `search_knowledge_base` call and a citation
Live demo network failure	Have a recorded fallback of the full flow
Small employee pool makes the assignment algorithm look trivial	Seed enough fake employees per department that load-balancing is visibly doing something
Scope creep into stretch features	Lock Section 2's MVP list, stretch items only after the core loop works end to end

---

19. What Makes This Different

Confidence decides the action instead of forcing a guess. Assignment is explainable with an actual score, not a black box. Access is tied to a live, earned tier instead of a static role field. Every correction and resolution feeds back into the system, so it improves the longer it runs. Load balancing is fairness-aware, not a naive "always pick the top match." And the user never

leaves one conversation, the human's reply lands in the same thread instead of a separate portal.

---

20. Related Documents

- `PS18_Algorithm_Design_Document.md` — full algorithm detail and case-by-case solutions
- `PS18_Team_Sprint_Plan_and_Orchestrator_Scaffold.md` — sprint phases, paste-ready system prompt and tool schema, demo checklist
- `PS18_Employee_User_Assignment_Plan.md` — the pitch-focused writeup of each system and the differentiators